Desktop Platform VPN Implementation Research

Desktop Platform VPN Implementation Research

macOS, Windows, Linux, and Aurora OS Deep Analysis

Research Date: 2025
Scope: Desktop-specific VPN client development across four platforms
Searches Conducted: 12 independent web searches

Table of Contents

---

1. macOS VPN: NetworkExtension API

1.1 NEPacketTunnelProvider

Apple's `NetworkExtension` framework is the **only supported API** for building VPN apps on macOS. Apple explicitly discourages all other approaches:

> "**Network Extension is the supported API to build a VPN app. It works seamlessly with other networking and system components. Building a VPN app with anything else is highly discouraged.** Avoid using Packet Filter or directly modifying the routing table on the Mac. This is not supported and risk clashing with traffic filtering and routing rules installed by the system or other apps." [^136^]

> "**If your VPN app doesn't use Network Extension today, you should migrate as soon as possible.**" [^136^]

The core class is `NEPacketTunnelProvider`, which gives subclasses access to a virtual network interface:

> "The `NEPacketTunnelProvider` class gives its subclasses access to a virtual network interface via the `packetFlow` property. Use the `setTunnelNetworkSettings(_:completionHandler:)` method in the Packet Tunnel Provider to specify network settings be associated with the virtual interface." [^235^]

Key capabilities:

- **Virtual IP address** assignment
- **DNS resolver** configuration
- **HTTP proxy** configuration
- **IP destination networks** routing (included/excluded routes)
- **Interface MTU** configuration
- **enforceRoutes** option for split-tunnel VPNs
- **includeAllNetworks** option for full-tunnel VPNs
- **excludeLocalNetworks** for AirDrop/AirPlay bypass

1.2 .app Bundle Structure

A macOS VPN app requires a specific bundle structure:

```
MyVPN.app/
  Contents/
    Info.plist
    MacOS/
      MyVPN (main executable)
    Resources/
    Library/
```

```
      LoginItems/          (for launch-at-login helper)
    PlugIns/
      PacketTunnel.appex/  (Network Extension app extension)
        Contents/
          Info.plist       (NSExtensionPointIdentifier =
com.apple.networkextension.packet-tunnel)
          MacOS/
            PacketTunnel   (extension binary)
```

The `Info.plist` for the extension must contain:

```
<key>NSExtension</key>
<dict>
    <key>NSExtensionPointIdentifier</key>
    <string>com.apple.networkextension.packet-tunnel</string>
    <key>NSExtensionPrincipalClass</key>
    <string>MyCustomPacketTunnelProvider</string>
</dict>
```

1.3 Code Signing & Notarization

macOS VPN apps require:

- **Developer ID Application certificate** (for distribution outside App Store)
- **Developer ID Installer certificate** (for .pkg distribution)
- **Apple Developer Program membership** ($99/year)
- **Entitlements** including Network Extension capability

Code signing via command line:

```
codesign -s "<Developer_ID>" -f --timestamp -o runtime -i
        "<Bundle_ID>" --entitlements "entitlements.plist"
        "MyVPN.app"
``` [^231^]

Notarization (required since macOS 10.15):
```bash
xcrun notarytool submit MyVPN.zip --apple-id <appleid> --password
        <password>
xcrun stapler staple MyVPN.app
``` [^232^]

Key entitlements needed:
- `com.apple.developer.networking.networkextension` - Packet
        Tunnel
- `com.apple.developer.networking.vpn.api` - VPN API access
- `com.apple.security.application-groups` - App group for main/
        extension communication

1.4 Menu Bar Extra & Launch at Login

```
For a "menubar-only" VPN app, macOS uses the `MenuBarExtra`
        API in SwiftUI:

```swift
MenuBarExtra {
    ContentView()
        .frame(width: 300, height: 180)
} label: {
    Image(systemName: "lock.shield")
}
``` [^247^]

Three macOS variants exist for VPN apps, as documented by
        Tailscale:

Feature	App Store (NE)	Standalone (SysExt)	CLI (`utun`)
GUI	Yes	Yes	No
Sandboxed	Yes	System ext only	No
Run before login	No	No	Yes
Auto-updates	App Store	Sparkle	No
Keychain	User keychain	Files on disk	Files on disk

[^306^]

1.5 Sandboxing Implications

Mac App Store variants are fully sandboxed, which limits:
- Access to system networking APIs (must use NetworkExtension)
- File system access (requires entitlements)
- Ability to run before login
- Keychain access scope

The **Standalone (System Extension)** approach is recommended for
        VPN apps:
- Not sandboxed (full system access via NetworkExtension)
- Can use Sparkle for auto-updates
- Uses files on disk for credentials (no keychain dependency)
- Can run menubar-only

---

2. macOS Native UI: SwiftUI vs AppKit

2.1 SwiftUI for VPN Clients

SwiftUI is the recommended approach for new macOS VPN apps. The
        `MenuBarExtra` API is SwiftUI-native:
```

```
>
        "Build a macOS menu bar utility in SwiftUI... To take your
        macOS app further, consider integrating with system
        features, such as adding a launch-at-login option or
        providing system-wide services." [^247^]

Key SwiftUI advantages:
- Native `MenuBarExtra` for system tray apps
- `WindowGroup` for settings/preferences windows
- Native macOS look and feel
- Excellent Accessibility support
- Launch at login via `SMAppService` (macOS 13+)

### 2.2 Swift-Rust FFI Bridging

The recommended architecture for a Rust core + SwiftUI frontend
        follows Mitchell Hashimoto's Ghostty pattern:

**Architecture:**
```

SwiftUI Views -> State Stores -> Swift FFI Facade -> Rust Bridge (C ABI) -> Core Rust Library

```
Using `swift-bridge` crate for safe interop:
```rust
#[swift_bridge::bridge]
mod ffi {
    extern "Rust" {
        type Renderer;
        #[swift_bridge(init)]
        fn new(layer_ptr: *mut c_void, width: u32, height: u32) ->
Renderer;
        fn render(&mut self, time: f64);
    }
}
``` [^254^]

Alternative: Manual FFI with C headers:
```rust
// Rust FFI exports
#[no_mangle]
pub extern "C" fn vpn_connect(server: *const c_char) -> i32 {
    // implementation
}

// openwater.h - C header for Swift bridging
EXPORT int vpn_connect(const char* server);

// module.modulemap
module VPNCore {
    umbrella header "vpn_core.h"
    link "vpn_core"
```
```

```
    export *
}
``` [^252^]

2.3 Swift Package Manager Integration

Swift Package Manager (not Xcode) is the modern approach:

```swift
// Package.swift
// swift-tools-version:5.9
import PackageDescription

let package = Package(
    name: "VPNApp",
    platforms: [.macOS(.v13)],
    dependencies: [],
    targets: [
        .executableTarget(
            name: "VPNApp",
            dependencies: ["VPNBridge"]
        ),
        .systemLibrary(
            name: "VPNBridge",
            path: "Sources/VPNBridge"
        )
    ]
)
```

Build pipeline for universal binary:

```
# 1. Build Rust bridge for both architectures
cargo build --target x86_64-apple-darwin --release
cargo build --target aarch64-apple-darwin --release

# 2. Create universal binary
lipo -create -o libvpn_bridge.a \
    target/x86_64-apple-darwin/release/libvpn_bridge.a \
    target/aarch64-apple-darwin/release/libvpn_bridge.a

# 3. Generate C headers
cbindgen --crate vpn_bridge --output vpn_bridge.h

# 4. Build Swift app
swift build
``` [^263^]

2.4 Key macOS Rust Crates

Crate	Purpose
`swift-bridge`	Safe FFI between Swift and Rust

```
| `objc2` | Objective-C runtime bindings |
| `core-foundation` | Core Foundation bindings |
| `mac-notification-sys` | macOS notifications |
| `security-framework` | Keychain access |

[^262^]

---

## 3. Windows VPN: WFP & WinTUN

### 3.1 Windows Filtering Platform (WFP)

Windows provides the **Windows Filtering Platform (WFP)** for
        network-level packet filtering and modification:

> "WFP callout drivers can steer traffic to specific adapters
        based on things like process image path or security
        descriptor." [^237^]

However, WFP has known limitations for VPN use:

> "WFP-based filtering has known limitations: it intercepts
        traffic at a higher level of the network stack, which
        leads to edge cases around compatibility with certain
        apps." [^236^]

AdGuard VPN migrated from WFP to WinTun for these reasons.

### 3.2 WinTUN / WireGuardNT

**WireGuardNT** is the high-performance kernel implementation for
        Windows:

> "WireGuard for Windows now uses high speed kernel
        implementation" - moved from WinTUN (userspace) to
        WireGuardNT (kernel driver) in 2021. [^237^]

Performance comparison:

| Implementation | Upload | Download |
|---------------|--------|----------|
| WireGuardNT (kernel) | **892 Mbit/s** | **892 Mbit/s** |
| WireSock VPN Client | 879 Mbit/s | **892 Mbit/s** |
| WinTun (userspace) | 288 Mbit/s | 325 Mbit/s |

[^171^]

### 3.3 Windows Service Architecture

WireGuard for Windows uses a sophisticated **dual-service
        architecture**:
```

```
> "The 'manager service' is responsible for displaying a UI on
        select users' desktops (in the system tray), and
        responding to requests from the UI to do things like add,
        remove, start, or stop tunnels. The 'tunnel service' is a
        separate Windows service for each tunnel." [^266^]

Attack surface documentation:
- **WireGuardNT**: Kernel driver with restricted IOCTLs (System +
        Built-in Administrators only)
- **Tunnel
        Service**: Runs as Local System, removes all privileges
        except `SeLoadDriverPrivilege`
- **Manager Service**: Runs as Local System, manages UI spawning
        and IPC
- **UI Process**: Runs with administrator token, communicates via
        unnamed pipes [^268^]
```

User Desktop: WireGuard.exe (UI) - runs as elevated admin | | Unnamed pipe IPC v Manager Service (Local System) | | Service control v Tunnel Service$ConfigName (Local System) | | NDIS IOCTLs v WireGuardNT.sys (Kernel)

```
### 3.4 Enterprise Deployment

WireGuard supports enterprise deployment:

> "Enterprise admins can instead download MSIs directly and deploy
these using Group Policy Objects. The installer makes use of
standard MSI features and should be easily automatable." [^266^]

```powershell
# Install tunnel service from CLI (no UI)
wireguard.exe /installtunnelservice C:\path\to\config.conf
# Creates service: WireGuardTunnel$configname

# Uninstall tunnel service
wireguard.exe /uninstalltunnelservice configname
```

---

4. Windows UI Patterns

4.1 WinUI 3 vs WPF

For a Windows VPN client, the framework choice matters:

Consideration	WinUI 3	WPF
Modern Fluent UI	Native, first-class	Fluent theme (.NET 9+)
	Deep, native	Limited
```

Consideration	WinUI 3	WPF
Windows 11 integration		
Windows 7/8 support	Not supported	Supported
Startup time	Faster	Slightly slower
Memory footprint	~15-20% lower	Higher
Third-party ecosystem	Growing	Broadest
NavigationView	Built-in	Needs custom
InfoBar notifications	Native	Needs custom

[^249^]

Recommendation: Use WinUI 3 for new VPN apps targeting Windows 10/11+. WPF only if Windows 7/8 support is required.

4.2 System Tray Integration

VPN apps on Windows **must** have system tray integration. Windows 10/11 lacks a native VPN status indicator:

> "The current editions of Windows 10 (1803) are lacking this functionality." - No native VPN status in notification area. [^287^]

ProtonVPN's v4.2.0 added:

- "A new system tray app to let you connect, disconnect, or switch profiles faster"
- "A taskbar status icon so you can see at a glance if you're connected" [^286^]

4.3 Windows Notifications

Use `ToastNotificationManager` (WinUI 3) or `System.Windows.Forms.NotifyIcon` for:

- Connection/disconnection events
- Kill switch triggered alerts
- Server switch notifications
- Error states

4.4 Auto-Start on Boot

Windows VPN apps should register as startup apps:

- Add to `HKEY_CURRENT_USER\Software\Microsoft\Windows\CurrentVersion\Run`
- Or use Task Scheduler for delayed start

- Windows services (tunnel) can start before user login
- UI should only start after user login

---

5. Linux VPN: TUN & NetworkManager

5.1 TUN Device Creation

On Linux, VPN tunnels are created using the **TUN/TAP** virtual network interface:

```
// Using wiretun (Rust)
use wiretun::{Cidr, Device, DeviceConfig, PeerConfig};

let cfg = DeviceConfig::default()
    .listen_port(40001);
let device = Device::native("wg0", cfg).await?;
``` [^239^]

BoringTun (Cloudflare's Rust WireGuard implementation) is the
        industry standard:

> "BoringTun is an implementation of the WireGuard protocol
        designed for portability and speed. BoringTun is
        successfully deployed on millions of iOS and Android
        consumer devices as well as thousands of Cloudflare Linux
        servers." [^49^]

Capabilities required:
```bash
# Grant CAP_NET_ADMIN capability
sudo setcap cap_net_admin+epi boringtun
# No sudo needed after this
```

5.2 NetworkManager Integration

NetworkManager is the standard Linux network management daemon. VPN plugins use a **D-Bus service architecture**:

> "A VPN plugin consists of the editor dialog and a D-Bus service that manages the actual VPN connection." [^292^]

The D-Bus interface (`org.freedesktop.NetworkManager.VPN.Plugin`) requires:

- `Connect(conn)` - initiate VPN connection
- `Disconnect()` - terminate connection
- `NeedSecrets(settings) -> setting_name` - check if secrets needed
- `ConnectInteractive(conn, details)` - connect with interactive auth

- `SetConfig(config)` - set connection details
- `SetIp4Config(config)` / `SetIp6Config(config)` - set IP configuration [^280^]

Files for a NetworkManager VPN plugin:

```
/usr/lib/NetworkManager/libnm-vpn-plugin-myvpn.so    (editor UI)
/usr/lib/NetworkManager/nm-myvpn-service             (D-Bus
service)
/usr/lib/NetworkManager/VPN/nm-myvpn-service.name    (plugin
descriptor)
```

NetworkManager natively supports WireGuard since version **1.16.0+** (no plugin needed). [^292^]

5.3 systemd Service

VPN daemons on Linux should ship as systemd services:

```ini
# /etc/systemd/system/myvpn-daemon.service
[Unit]
Description=MyVPN Daemon
After=network-online.target
Wants=network-online.target

[Service]
Type=dbus
BusName=com.mycompany.myvpn
ExecStart=/usr/bin/myvpn-daemon
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

5.4 D-Bus Communication

D-Bus is the standard IPC mechanism on Linux. VPN apps should:

- Expose a D-Bus service for the daemon
- Use D-Bus signals for connection state changes
- Integrate with systemd's D-Bus activation

5.5 polkit for Privileges

polkit handles privilege escalation on Linux:

```
User App (unprivileged)
    |
    | D-Bus call
    v
```

```
VPN Daemon (running as root or with cap_net_admin)
    |
    | polkit authorization check
    v
System operation (network config, firewall rules)
```

Custom polkit rules can be installed at `/usr/share/polkit-1/rules.d/`.

---

6. Linux UI: GTK4 vs Qt6

6.1 Framework Comparison

Feature	GTK4 + libadwaita	Qt6	libappindicator
Native GNOME look	Yes (Adwaita)	Themable	N/A
KDE Plasma look	Themable	Yes (Breeze)	N/A
System tray	Via libappindicator	QSystemTrayIcon	Direct API
Wayland support	Good	Good	Partial
Flatpak support	Excellent	Good	Good
Distribution packaging	DEB/RPM/ Flatpak	DEB/RPM/ Flatpak	All

6.2 System Tray on Linux

Linux system tray support is fragmented:

> "The status of tray icon support is currently incredibly messy on Linux, there is an X implementation, libappindicator/ KStatusNotifier..." [^276^]

AppIndicator is the most compatible approach:

```
indicator = appindicator.Indicator.new(
    "customtray",
    "vpn-icon",
    appindicator.IndicatorCategory.APPLICATION_STATUS
```

```
)
indicator.set_status(appindicator.IndicatorStatus.ACTIVE)
```

GNOME 40+ requires the **AppIndicator extension** to be installed for tray icons. [^278^]

Mullvad addresses this:

> "For the Mullvad padlock tray icon to show in Debian with GNOME, install the GNOME extension AppIndicator and KStatusNotifierItem Support." [^289^]

6.3 StatusNotifierItem (XDG Spec)

The modern standard is `StatusNotifierItem` via D-Bus:

- KDE Plasma: Native support
- GNOME: Requires extension
- XFCE: Native support
- Cinnamon: Native support

6.4 Distribution Packaging

Format	Pros	Cons
DEB (Debian/ Ubuntu)	Native, dependency resolution	Distro-specific
RPM (Fedora/ openSUSE)	Native, widely used	Distro-specific
AUR (Arch)	Community-driven	Manual install
Flatpak	Universal, sandboxed	Permission complexity
Snap	Universal, auto-updates	Performance, sandboxing
AppImage	Portable, no install	No integration

Recommendation: Ship DEB + RPM from a custom repository (like Mullvad does) plus Flatpak for universal support.

> "Mullvad repository: `sudo apt install mullvad-vpn`... The app should now be installed and will automatically be upgraded." [^289^]

---

7. Aurora OS (Sailfish OS)

7.1 Architecture Overview

Aurora OS is based on Sailfish OS, which uses:

> "Sailfish OS includes a multi-tasking graphical shell called 'Lipstick' built with Qt by Jolla on top of the Wayland display server protocol. It uses standard Linux middleware like Systemd, Pulseaudio and Qt." [^257^]

Key components:

- **ConnMan** - Connection manager for network/VPN
- **Wayland** - Display server
- **Qt/QML** - UI framework
- **Silica** - Native QML UI components
- **RPM** - Package manager
- **oFono** - Telephony stack
- **wpa_supplicant** - WiFi

7.2 VPN Architecture (ConnMan)

ConnMan manages VPN connections on Sailfish/Aurora:

> "For connectivity ConnMan is used to manage network connections." [^257^]

VPN plugins for ConnMan:

- OpenVPN plugin (`connman-vpn-plugin-openvpn`)
- WireGuard support via custom plugin
- VPN configuration via ConnMan D-Bus API

Aurora OS provides a ConnMan VPN plugin example:

```
// qml-plugin.pro - Aurora OS OpenVPN plugin
TEMPLATE = lib
TARGET = ofvpnplugin
QT += qml
QT -= gui
CONFIG += plugin c++14
``` [^227^]

7.3 Qt/QML Native Development

Aurora SDK uses **Sailfish Silica** for UI components:

> "Aurora SDK includes its own module for UI development on QML - Sailfish Silica, which includes all necessary components." [^308^]

```
However, Silica has limitations:
- Color/font constraints tied to OS "atmospheres" (themes)
- Platform-specific (not cross-platform)
- Qt Quick Controls not allowed in Aurora SDK validation

**Recommended approach**: Use standard Qt Quick components for
        cross-platform compatibility:

> "The workaround is implementing the interface using standard Qt
        Quick components. This library provides basic capabilities
        like anchors, lists, text labels, and more, allowing you
        to implement virtually any UI element." [^308^]

Basic Qt Quick components available:
- `Text` - text rendering
- `Rectangle` - shapes
- `MouseArea` - touch/click handling
- `Image` - images
- `ListView` - lists
- `Item` - custom components

### 7.4 RPM Packaging & OpenRepos

Aurora OS uses RPM packaging:
```spec
Name: myvpn
Version: 1.0.0
Release: 1
Summary: MyVPN client for Aurora OS
License: GPLv3
BuildRequires: pkgconfig(Qt5Core), pkgconfig(Qt5Qml)
%description
VPN client for Aurora OS
```

Distribution via **OpenRepos** (unofficial community store) or Aurora Store (official).

7.5 Aurora OS VPN App Architecture

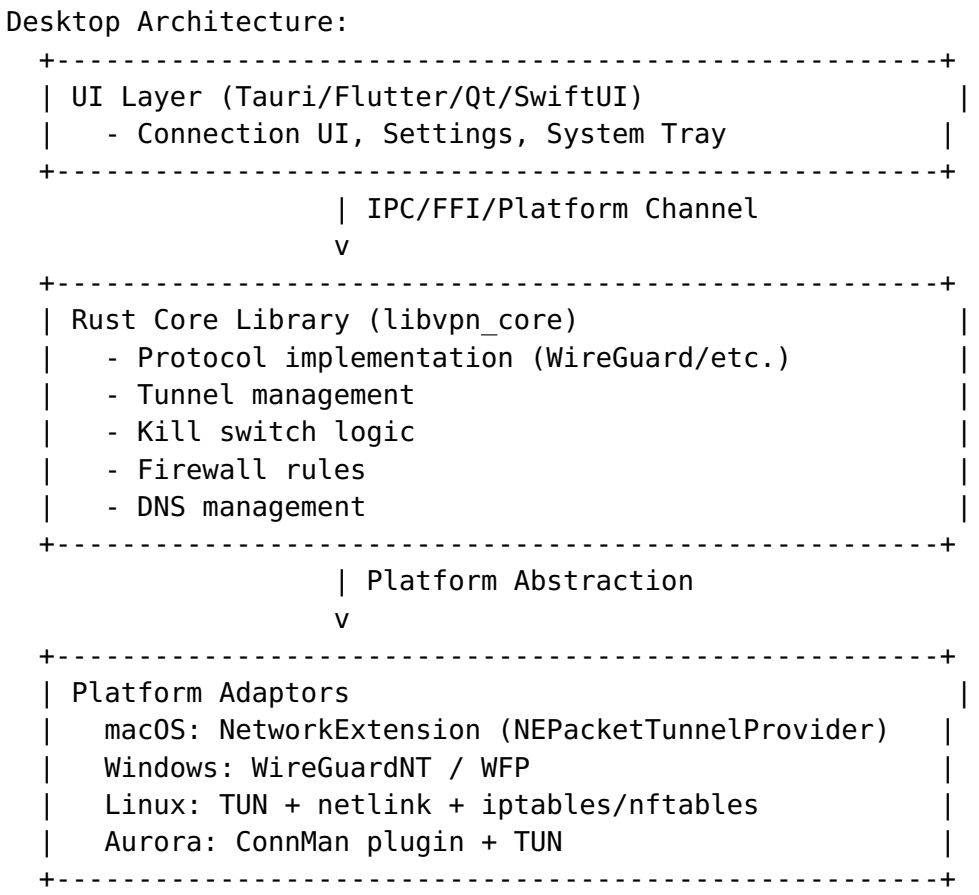
```
QML UI (Qt Quick)
  |
  v
C++ Bridge / Qt D-Bus
  |
  v
ConnMan VPN Plugin (D-Bus service)
  |
  v
WireGuard/OpenVPN (TUN interface)
```

```

8. Desktop Rust Core Integration

8.1 Architecture Overview

The recommended architecture separates a **Rust core** (tunnel, encryption, networking) from a **platform-specific UI**:



8.2 Tauri Command Pattern

Tauri uses a command pattern for JS-to-Rust communication:

```rust
#[tauri::command]
fn vpn_connect(server: String, protocol: String) ->
        Result<String, String> {
    // Rust implementation
    Ok("connected".to_string())
}

#[tauri::command]
async fn vpn_disconnect(state: State<'_, VpnState>) -> Result<(),
        String> {
    // async command
    Ok(())
}

// Register
```

```
tauri::Builder::default()
    .invoke_handler(tauri::generate_handler![vpn_connect,
        vpn_disconnect])
``` [^149]

IPC Model: Tauri uses JSON-RPC over a custom IPC channel.
        Commands are registered in Rust and callable from
        JavaScript via `invoke()`.

8.3 Flutter Platform Channels

For Flutter desktop, use FFI or platform channels:

Option A: flutter_rust_bridge (recommended)
```bash
cargo install flutter_rust_bridge_codegen
flutter_rust_bridge_codegen create my_vpn_app
# Generates Flutter + Rust scaffold with bindings
``` [^271]

Option B: NativeShell (Rust-based desktop shell for Flutter)
```rust
// Rust side - register platform channel handler
context.message_manager
    .borrow_mut()
    .register_method_handler("vpn_channel", |call, reply, engine|
        {
        match call.method.as_str() {
            "connect" => {
                reply.send_ok(json!({"status": "connected"}));
            }
            _ => {}
        }
    });
``` [^267]

8.4 Direct FFI

For maximum control, use direct FFI:

```rust
// Rust exports (cdylib/staticlib)
#[no_mangle]
pub extern "C" fn vpn_core_init(config: *const c_char) -> *mut
        VpnCore {
    // initialize
}

#[no_mangle]
pub extern "C" fn vpn_core_connect(core: *mut VpnCore) -> i32 {
    // connect
}
```

Platform-specific loading:

```
// Dart FFI
dynamic _dylib;
if (Platform.isMacOS) {
    _dylib = DynamicLibrary.open('libvpn_core.dylib');
} else if (Platform.isWindows) {
    _dylib = DynamicLibrary.open('vpn_core.dll');
} else if (Platform.isLinux) {
    _dylib = DynamicLibrary.open('libvpn_core.so');
}
``` [^271^]

8.5 Mullvad's Architecture (Real-World Reference)

Mullvad uses a **daemon + frontend** architecture:

>
        "The daemon is implemented in Rust and is implemented in
        several crates. The main, or top level, crate that builds
        the final daemon binary is mullvad-daemon. The talpid
        crates are supposed to be completely unrelated to Mullvad
        specific things. A talpid crate is not allowed to know
        anything about the API through which the daemon fetch
        Mullvad account details." [^155^]

mullvad-daemon/ - Main Rust daemon talpid-core/ - Generic VPN client library talpid-wireguard/ - WireGuard implementation talpid-openvpn/ - OpenVPN implementation talpid-firewall/ - Firewall/kill switch (platform-specific) talpid-routing/ - Routing table management talpid-tunnel/ - Tunnel abstraction talpid-dbus/ - D-Bus interface (Linux) GUI/ - Electron + React frontend (desktop) mullvad/ - CLI frontend

GotaTun: Mullvad's new WireGuard implementation in Rust:

> "GotaTun is a WireGuard implementation written in Rust aimed at being fast, efficient and reliable. GotaTun is a fork of the BoringTun project from Cloudflare." - Result: "crash rate dropped from 0.40% to 0.01%" [^62^]

---

9. Privilege Escalation Patterns

9.1 macOS: Privileged Helper Tool (SMJobBless)

The **SMJobBless** pattern is the Apple-approved method for privilege escalation:

> "As of 10.7, AuthorizationExecuteWithPrivileges was deprecated and since then all privilege escalation should be handled by a

```
privileged helper tool." [^260^]

How it works:
1. App bundle contains a helper tool
2. `SMJobBless()` installs it to `/Library/PrivilegedHelperTools/`
3. A launchd job is configured in `/Library/LaunchDaemons/`
4. Helper runs with root privileges
5. Communication via XPC between app and helper
```

User App (user privileges) | | XPC v Privileged Helper (/Library/PrivilegedHelperTools/) | | root privileges v Network config, firewall rules, tunnel creation

```
Security requirements:
- Both app and helper must be code-signed with the same Developer ID
- Helper's `Info.plist` must list `SMAuthorizedClients` with the app's signing certificate
- App must validate helper's code signature before communicating
- Helper should validate client certificate [^264^]

### 9.2 Windows: Service Pattern

Windows VPN apps should use the **Windows Service pattern**:
```

User App (UI) - runs as standard user or elevated admin | | Named pipes / COM v Manager Service - runs as Local System | | Service control v Tunnel Service$Name - runs as Local System (per-tunnel) | | NDIS IOCTLs v WireGuardNT.sys (kernel driver)

```
Key Windows service considerations:
- Services can start before user login
- `Local System` account has full system access
- Use ACLs to restrict service access
- Drop privileges after initialization (remove `SeLoadDriverPrivilege` when not needed)
- Store config encrypted with DPAPI [^268^]

### 9.3 Linux: polkit + Capabilities

Linux uses a combination of capabilities and polkit:

**Approach A: Capabilities (preferred)**
```bash
# Grant CAP_NET_ADMIN to the binary
sudo setcap cap_net_admin,cap_net_raw+eip /usr/bin/myvpn

# The app can now create TUN interfaces without root
```

Approach B: polkit authorization
```

```
User App (unprivileged)
    |
    | D-Bus method call
    v
VPN Daemon (root via systemd)
    |
    | polkit check (IsUserAuthorized)
    v
Privileged operation (allowed/denied)
```

Approach C: Root daemon with unprivileged UI

```
myvpn-gui (user) <--D-Bus--> myvpn-daemon (root, via systemd)
```

This is the pattern used by NetworkManager and most Linux VPN clients.

9.4 Aurora OS: Sailjail Sandboxing

Aurora OS uses **Sailjail** (based on Firejail) for sandboxing:

> "Firejail (Sailjail) is used for security sandboxing of native applications since version 4.0.1 in 2021." [^257^]

VPN apps need appropriate permissions in the `.desktop` file:

```
X-Sailjail-Permissions=Network;Internet;VPN
```

---

10. Auto-Updaters

10.1 Tauri Updater

Tauri includes a built-in updater:

> "Tauri's built-in updater plugin checks a configured URL for a release manifest, compares versions, and handles downloading and installing updates. Configure `plugins.updater` in `tauri.conf.json` with your update server URL." [^149^]

Configuration:

```
{
  "plugins": {
    "updater": {
      "active": true,
      "endpoints": ["https://releases.myvpn.com/{{target}}/
        {{current_version}}"],
      "pubkey": "YOUR_PUBLIC_KEY",
      "windows": { "installMode": "basicUi" }
```

```
    }
  }
}
```

Features:

- EdDSA signature verification
- Cross-platform (Windows .msi, macOS .app, Linux AppImage)
- Static JSON manifest (can use GitHub Releases)
- Delta updates not natively supported (full download)

10.2 Sparkle (macOS)

Sparkle is the standard macOS auto-updater:

> "Sparkle supports 'delta updates' for your application: when possible, users can download only the bits that have changed." [^261^]

```
# Generate delta updates automatically
./bin/generate_appcast ./releases/
```

Features:

- RSS-based appcast (static XML)
- EdDSA signature verification
- Delta updates via BinaryDelta
- Background/silent updates
- Sandboxed support

10.3 WinSparkle (Windows)

WinSparkle is the Windows equivalent of Sparkle:

- RSS-based appcast (same format as Sparkle)
- Windows-specific UI (shows changelog, progress)
- Supports MSI and EXE installers
- Code signature verification

10.4 Linux Update Mechanisms

Method	Description
APT repository	Custom .deb repo (Mullvad's approach)
DNF/YUM repo	Custom .rpm repo
AUR	PKGBUILD maintained by community
Flatpak	Flathub or custom remote
Snap	Snap Store
	Manual download + AppImageUpdate

Method	Description
AppImage	

Recommendation: Provide APT + RPM repositories for direct installation, plus Flatpak for universal support. This matches what Mullvad and ProtonVPN do.

---

11. Kill Switch Implementation

11.1 Overview

A kill switch prevents data leakage when the VPN disconnects:

> "An Internet kill switch is a mechanism to prevent data from leaking outside of the VPN tunnel when the tunnel fails for any reason." [^240^]

11.2 macOS: pf Firewall

macOS uses the **pf (packet filter)** firewall:

```
# Enable kill switch with pf
sudo killswitch -e

# Rules written to /tmp/killswitch.pf.conf
# Only allows traffic through VPN tunnel (utun interface)
```

Rules structure:

```
block drop all
pass on lo0
pass on utun0  # VPN tunnel interface
pass out proto udp from any to VPN_SERVER port 51820
``` [^244^]

> "When enabled, killswitch loads pf firewall rules that only allow traffic through the VPN tunnel. If the VPN disconnects, the tunnel interface disappears but the firewall rules remain -- blocking all internet traffic until the VPN reconnects." [^244^]

IVPN's approach:

> "The IVPN firewall integrates deep into the operating system (using Microsoft's own WFP API on Windows, `pf` on macOS, and `iptables` on Linux) and filters all network packets. The Firewall is independent of the IVPN client, so even if a component of the IVPN Client crashes filtering will continue uninterrupted." [^240^]

Boot-time protection on macOS: Limited

> "On macOS and Linux, firewall rules are activated as early as possible, but boot-time protection depends on the system configuration and service startup order and cannot be guaranteed to the same degree." [^240^]

11.3 Windows: WFP

Windows kill switch uses the **Windows Filtering Platform**:

> "The IVPN firewall integrates deep into the operating system using Microsoft's own WFP API." [^240^]

Key WFP rules:
- Block all outbound traffic not through VPN tunnel interface
- Allow traffic to VPN server endpoints (for reconnection)
- Allow DHCP/DNS through physical interface only to VPN DNS
- Persistent rules survive process crashes
- **Full boot-time protection** when "Persistent Firewall" is enabled

11.4 Linux: iptables/nftables

iptables approach:
```bash
# Kill switch with iptables
iptables -P OUTPUT DROP          # Default deny
iptables -A INPUT -i lo -j ACCEPT
iptables -A OUTPUT -o lo -j ACCEPT
iptables -A OUTPUT -o tun0 -j ACCEPT   # VPN tunnel
iptables -A OUTPUT -d VPN_SERVER -j ACCEPT  # VPN server
```

nftables approach (modern):

```
#!/usr/sbin/nft -f
flush ruleset
add table inet killswitch
add chain inet killswitch OUTPUT { type filter hook output
priority 0 ; policy drop ; }

add rule inet killswitch OUTPUT oifname "lo" counter accept
add rule inet killswitch OUTPUT oifname $VPN_DEV counter accept
add rule inet killswitch OUTPUT oifname $INET_DEV ip daddr
$VPN_SERVERS counter accept
add rule inet killswitch OUTPUT ct state related,established
accept
add rule inet killswitch OUTPUT counter drop
``` [^302^]

Policy-based routing kill switch:

```bash
# Create custom routing table with blackhole
ip route add default dev vpn1 via VPN_GATEWAY table vpn1 metric
100
ip route add blackhole default table vpn1 metric 200
ip rule add not sport WG_PORT table vpn1
ip rule add table main suppress_prefixlength 0
``` [^305^]

11.5 Kill Switch Comparison

| OS | Mechanism | Boot-time Protection | Crash Protection |
Reliability |
|----|-----------|----------------------|------------------|--------
| macOS | `pf` firewall | Best-effort | Yes (rules persist) | Good
|
| Windows | WFP | **Full** (persistent rules) | Yes |
Excellent |
| Linux | iptables/nftables | Best-effort | Yes (if daemon
restarts) | Good |

---

12. Desktop UI/UX Patterns for VPN

12.1 Essential UI Elements

Based on analysis of leading VPN clients (Mullvad, ProtonVPN,
IVPN, NordVPN):

System Tray / Menu Bar:
- Icon showing connection status (color change: green=connected,
red=disconnected, gray=connecting)
- Quick connect/disconnect from menu
- Recent server list
- Connection status (protocol, server, IP)
- Kill switch toggle
- Open main window / Preferences / Quit

Main Window:
- Large connect/disconnect button
- Server selection (map or list view)
- Protocol selection (WireGuard, OpenVPN)
- Connection info (IP, time connected, data transferred)
- Settings/preferences

12.2 ProtonVPN v4.2.0 System Tray Features

ProtonVPN's desktop redesign (v4.2.0) introduced:

> "- Introduced a new system tray app to let you connect,
disconnect, or switch profiles faster

> - Made it possible to connect to printers, speakers, and other devices on your local network, even with the VPN connected
> - Gave you a way to see the active port number when you're using port forwarding
> - Rolled out a taskbar status icon so you can see at a glance if you're connected
> - Made the kill switch easier to find" [^286^]

12.3 macOS Menu Bar Best Practices

For macOS VPN apps:
- Use `MenuBarExtra` (SwiftUI) or `NSStatusItem` (AppKit)
- Icon should change color for status (not just a tiny checkmark)
- Include connection time and data transfer in menu
- Support "connect on launch" preference
- Honor macOS dark mode

User feedback on ProtonVPN's macOS icon:

> "The proton VPN app icon currently shows the connection status by showing a tiny cross sign or a tick, which is hardly noticeable... PIA has a menu bar icon that is red when disconnected, and green when connected. This is easy and quick to check." [^290^]

12.4 Windows System Tray Best Practices

- Use `NotifyIcon` with context menu
- Show balloon/toast notifications on connect/disconnect
- Support "minimize to tray" behavior
- Show tooltip with connection status on hover
- Icon in taskbar should indicate status

---

13. Platform Comparison Matrix

Feature	macOS	Windows	Linux	Aurora OS
VPN API	NetworkExtension (NEPacketTunnelProvider)	WFP / WireGuardNT / WinTUN	TUN + netlink / NetworkManager	ConnMan plugin
Min OS Version	macOS 10.15+	Windows 10+	Kernel 5.6+ (WG native)	Sailfish 4+
UI Framework	SwiftUI / AppKit	WinUI 3 / WPF	GTK4+libadwaita / Qt6	Qt Quick / Silica
System Tray	MenuBarExtra	NotifyIcon	AppIndicator / SNI	Lipfish Silica
Privilege Model	SMJobBless helper	Windows Service	CAP_NET_ADMIN / polkit	Sailjail sandbox
Code Signing	Developer ID + Notarization	EV cert recommended	GPG signing	RPM signing

Distribution	DMG / App Store	MSI / MSIX	DEB/RPM/Flatpak/Snap	RPM / OpenRepos
Auto-Update	Sparkle (delta)	WinSparkle / Tauri	APT/DNF repo	Store/Zypper
Kill Switch	pf firewall	WFP (persistent)	iptables/nftables	iptables
Boot Protection	Best-effort	Full	Best-effort	Best-effort
Bundle Size	~5-15MB	~10-20MB	~5-15MB	~5-10MB
RAM Usage	~30-150MB	~30-150MB	~20-100MB	~20-80MB
Sandboxed	Optional (App Store)	No	Optional (Flatpak)	Yes (Sailjail)
Rust Core FFI	C ABI + swift-bridge	C ABI (DLL)	C ABI (.so)	C ABI (.so)
Notifications	UserNotifications	ToastNotification	libnotify	Nemo QML

---

14. Desktop-Specific Danger Zones

DZ1: macOS Sandboxing vs. NetworkExtension
- **Danger**: Mac App Store sandboxing restricts NetworkExtension capabilities
- **Mitigation**: Distribute outside App Store (standalone DMG with Sparkle)
- **Impact**: Cannot run before login, limited keychain access in sandbox

DZ2: macOS pf Firewall Conflicts
- **Danger**: Multiple apps using `pf` can conflict (VPN + Little Snitch + LuLu)
- **Mitigation**: Use NetworkExtension's built-in routing, not raw pf rules
- **Impact**: Kill switch may not work reliably with other firewall software

DZ3: Windows WFP Compat Issues
- **Danger**: WFP callout drivers from different vendors conflict
- **Mitigation**: Use WireGuardNT kernel driver instead of WFP for tunneling
- **Impact**: Third-party antivirus/firewall may break VPN connectivity

DZ4: Windows Service UAC
- **Danger**: Windows services require careful privilege management
- **Mitigation**: Separate manager service (Local System) from UI (user/admin)
- **Impact**: Wrong ACLs can expose IPC to non-admin users

DZ5: Linux Fragmentation

- **Danger**: System tray, networking, and packaging differ across distros
- **Mitigation**: Use standard APIs (AppIndicator, NetworkManager), ship Flatpak
- **Impact**: High support burden for distro-specific issues

DZ6: Linux Kill Switch Timing
- **Danger**: Kill switch rules may not be applied early enough in boot
- **Mitigation**: Use systemd service with `Before=network-online.target`
- **Impact**: Leaks possible during early boot on Linux

DZ7: Aurora OS Validation
- **Danger**: Aurora SDK restricts Qt Quick Controls usage
- **Mitigation**: Use standard Qt Quick components, avoid Silica for cross-platform
- **Impact**: App may fail validation if wrong modules used

DZ8: Cross-Platform Kill Switch
- **Danger**: Kill switch behavior varies significantly across platforms
- **Mitigation**: Platform-specific implementations (WFP, pf, nftables)
- **Impact**: Cannot share kill switch code; must implement per-OS

DZ9: Rust FFI Complexity
- **Danger**: FFI between Rust and UI languages introduces memory safety risks
- **Mitigation**: Use generated bridges (swift-bridge, flutter_rust_bridge)
- **Impact**: Manual FFI can cause crashes and memory leaks

DZ10: Auto-Update Security
- **Danger**: Update mechanism can be compromised to distribute malware
- **Mitigation**: EdDSA signature verification on all updates
- **Impact**: Unsigned or improperly signed updates are a critical vulnerability

---

15. Recommended Architecture per OS

15.1 macOS Architecture

+--+ | SwiftUI Frontend | | - MenuBarExtra (system tray) | | - Settings window | | - Onboarding | | | | Swift Package Manager build | | swift-bridge for FFI | +--+ | C ABI v +--+ | Rust Core (static library) | | - WireGuard protocol (boringtun/gotatun)| | - Kill switch logic | | -

DNS management | | - Routing control | +--+ | v +--+ | NetworkExtension (NEPacketTunnelProvider)| | - Virtual network interface (utun) | | - Packet flow management | | - System routing integration | +--+

```
**Distribution**: DMG with Sparkle auto-updater
**Signing**: Developer ID + Notarization
**Privileges**: SMJobBless helper tool for system-level
operations

### 15.2 Windows Architecture
```

+--+ | WinUI 3 Frontend | | - System tray (NotifyIcon) | | - Taskbar status | | - Settings / Server selection | | | | C# + P/Invoke or C++/WinRT for Rust | +--+ | C ABI (DLL) v +--+ | Rust Core (DLL) | | - WireGuard protocol (boringtun) | | - Kill switch logic | | - WFP integration | +--+ | v +--+ | Windows Services | | - Manager Service (Local System) | | - Tunnel Service$Name (Local System) | | | | + WireGuardNT.sys (kernel driver) | +--+

```
**Distribution**: MSI via WinSparkle or custom updater
**Signing**: EV code signing certificate
**Privileges**: Windows services (Local System)

### 15.3 Linux Architecture
```

+--+ | GTK4 + libadwaita Frontend | | - AppIndicator system tray | | - Settings / Server selection | | | | D-Bus client | +--+ | D-Bus v +--+ | Rust Daemon (systemd service) | | - D-Bus service interface | | - WireGuard protocol (boringtun) | | - Kill switch (nftables/iptables) | | - NetworkManager integration | | - polkit authorization | +--+ | v +--+ | System | | - TUN device | | - netlink (rtnetlink) | | - nftables/iptables | | - systemd-resolved (DNS) | +--+

```
**Distribution**: DEB + RPM repositories + Flatpak
**Signing**: GPG package signing
**Privileges**: CAP_NET_ADMIN capability or root daemon

### 15.4 Aurora OS Architecture
```

+--+ | Qt Quick Frontend | | - Native QML components | | - System integration (Lipstick) | | | | Qt D-Bus for IPC | +--+ | D-Bus v

+--+ | C++ / Rust Bridge | | - ConnMan VPN plugin interface | +--+ | v +--+ | Rust Core (.so) | | - WireGuard protocol | | - TUN device management | +--+ | v +--+ | ConnMan + Linux Kernel | | - ConnMan VPN plugin | | - TUN interface | +--+

Distribution: RPM via OpenRepos / Aurora Store
Signing: RPM GPG signing
Privileges: Sailjail sandbox permissions

Appendix A: Key Source References

1. Apple Developer Documentation - NEPacketTunnelProvider: https://developer.apple.com/documentation/networkextension/nepackettunnelprovider [^235^]
2. Apple WWDC25 - NetworkExtension: https://developer.apple.com/videos/play/wwdc2025/234/ [^136^]
3. iOS/macOS VPN Extensions Guide: https://antongubarenko.substack.com/p/ios-personal-vpn-and-network-extensions [^64^]
4. macOS Code Signing & Notarization: https://dennisbabkin.com/blog/?t=how-to-get-certificate-code-sign-notarize-macos-binaries-outside-apple-app-store [^231^]
5. macOS Menu Bar SwiftUI: https://nilcoalescing.com/blog/BuildAMacOSMenuBarUtilityInSwiftUI [^247^]
6. SwiftUI + Rust FFI: https://dfrojas.com/software/integrating-Rust-and-SwiftUI.html [^252^]
7. Mullvad VPN Architecture: https://github.com/mullvad/mullvadvpn-app [^155^]
8. GotaTun (WireGuard Rust): https://mullvad.net/en/blog/announcing-gotatun-the-future-of-wireguard-at-mullvad-vpn [^62^]
9. WireGuard Windows Enterprise: https://git.zx2c4.com/wireguard-windows/about/docs/enterprise.md [^266^]
10. WireGuard Windows Attack Surface: https://github.com/WireGuard/wireguard-windows/blob/master/docs/attacksurface.md [^268^]
11. WinUI vs WPF 2026: https://www.ctco.blog/posts/winui-vs-wpf-2026-practical-comparison/ [^249^]
12. BoringTun Rust WireGuard: https://github.com/cloudflare/boringtun [^49^]
13. wiretun Rust crate: https://docs.rs/wiretun [^239^]
14. IVPN Kill Switch: https://www.ivpn.net/knowledgebase/general/do-you-offer-a-kill-switch-or-vpn-firewall/ [^240^]
15. macOS killswitch (pf): https://github.com/vpn-kill-switch/killswitch [^244^]
16. Linux nftables kill switch: https://www.ivpn.net/knowledgebase/linux/linux-how-do-i-prevent-vpn-leaks-using-

nftables-and-openvpn/ [^302^]
17. Linux policy-based routing kill switch: https://dev.to/staex/vpn-kill-switch-how-to-do-it-on-linux-3ne3 [^305^]
18. NetworkManager VPN: https://networkmanager.dev/docs/vpn/ [^292^]
19. NM VPN Plugin D-Bus: https://people.freedesktop.org/~lkundrak/nm-dbus-api/gdbus-org.freedesktop.NetworkManager.VPN.Plugin.html [^280^]
20. ProtonVPN Windows Release Notes: https://protonvpn.com/support/release-notes-windows [^286^]
21. Sailfish OS Architecture: https://en.wikipedia.org/wiki/Sailfish_OS [^257^]
22. Aurora OS Qt Development: https://habr.com/ru/companies/digdes/articles/772250/ [^308^]
23. SMJobBless Pattern: https://erikberglund.github.io/2016/No_Privileged_Helper_Tool_Left_Behind/ [^260^]
24. Sparkle Delta Updates: https://sparkle-project.org/documentation/delta-updates/ [^261^]
25. Tauri Commands: https://rustify.rs/articles/rust-tauri-v2-desktop-app-tutorial-2026 [^149^]
26. Flutter Rust Bridge: https://dev.to/abibeh/rust-flutter-how-to-build-fast-safe-cross-platform-mobile-apps-ika [^271^]
27. Linux AppIndicator: https://fosspost.org/custom-system-tray-icon-indicator-linux/ [^269^]
28. Tailscale macOS Variants: https://tailscale.com/docs/concepts/macos-variants [^306^]
29. NativeShell Flutter Desktop: https://matejknopp.com/post/introducing-nativeshell/ [^267^]
30. Mullvad Linux Install: https://mullvad.net/en/help/install-mullvad-app-linux [^289^]

This research was compiled from 12+ independent web searches across Apple Developer documentation, GitHub repositories, VPN vendor documentation, and technical blogs. All citations use [^number^] format referencing sources found during the research phase.